DashyDashboard

Screenshot Attestation — Setup & Operations Guide

Screenshot storage, e-mail notifications, and database migrations

Broadridge Financial Solutions

June 2026

Document type:	Setup / Operations Supplement
Companion to:	DashyDashboard Deployment Guide
Audience:	Server Administrators & DBAs
Classification:	Internal

Contents

Chapter 1

Overview

This guide is a supplement to the main *DashyDashboard Deployment Guide*. It covers the operational steps required to enable the **screenshot attestation** feature and the associated **e-mail notifications** introduced in the June 2026 release. Read the main deployment guide first; this document only describes what is *new*.

1.1 What changed

- Associates now attach a screenshot to each tool they used during a cycle. Reviewers (managers, GFH/GFH-Delegate, and Admin) approve or reject each screenshot.
- Screenshots are written to a disk folder *outside* the deployed application directory.
- Two new configuration sections — `Screenshots` and `Email` — must be present in `appsettings.Production.json` (or supplied via environment variables).
- Two new Entity Framework migrations add columns to the database.
- Optional event-driven e-mail (rejected screenshot / all approved) requires an internal SMTP relay and an approved sending mailbox.

1.2 What an administrator must do (checklist)

1. Add the `Screenshots` and `Email` sections to `appsettings.Production.json`, or set the equivalent environment variables (Chapter 2).
2. Create the screenshot storage folder outside the app folder and grant the IIS app-pool identity *Modify* access (Chapter 3).
3. Apply the two database migrations, in order (Chapter 4).
4. (Optional) Request the internal SMTP relay details and an approved sending mailbox, then enable e-mail (Chapter 5).
5. Confirm the storage folder is covered by server backups and understand the retention policy (Chapter 6).

Important

`dotnet publish` OVERWRITES `appsettings.Production.json` and the entire app folder on every deployment. The screenshot folder must therefore live *outside* the app folder, and the `Screenshots/Email` configuration must be re-applied (or supplied via environment variables, which survive a republish) after *every* publish. See Chapter 2.

Chapter 2

Configuration Keys

The application reads two new configuration sections. The base `appsettings.json` ships with safe defaults (storage on `C:`, e-mail disabled), but production should override at least `Screenshots:RootPath` and, if mail is wanted, the `Email` section.

2.1 The `Screenshots` section

Key	Meaning
`Screenshots:RootPath`	Absolute disk path under which all screenshots are stored, using the layout `{root}\{cycleId}\{associateId}\{clientId}\{toolId}.webp` (plus a `_thumb.webp` thumbnail). **Must be OUTSIDE the deployed app folder** (Chapter 3). The IIS app-pool identity needs *Modify* permission here.
`Screenshots:RetainCycles`	Integer number of recent cycles to keep on disk. Default `6`. Retention is a policy target only — the automatic sweep is a future job (Chapter 6).

2.2 The `Email` section

Key	Meaning
`Email:Enabled`	Master switch. `false` (the default) means no mail is ever sent — the request paths run normally, mail is simply skipped. Set `true` only once the relay and sender are confirmed.
`Email:SmtpHost`	Hostname of the internal SMTP relay. The relay is contacted *unauthenticated* and without TLS (corporate internal relay).
`Email:SmtpPort`	Relay port. Default `25`.
`Email:From`	The sending mailbox address that appears in the *From* header. Must be an approved sender on the relay.

Key	Meaning
`Email:AppUrl`	Absolute base URL of the deployed app (e.g. `http://clipvwbpod02/dashydashboard/`). Used to build the "open DashyDashboard" links in mail bodies.
`Email:Events:ScreenshotRejected`	Per-event toggle for the "your screenshot was rejected" mail to the associate. Default `true` (gated by the master switch).
`Email:Events:AllApproved`	Per-event toggle for the "all your screenshots are approved" mail. Sent exactly once, on the not-complete to complete transition. Default `true`.

2.3 Example `appsettings.Production.json` fragment

Add these two sections alongside the existing `ConnectionStrings` block:

```
"Screenshots": {
    "RootPath": "D:\\DashyData\\Screenshots",
    "RetainCycles": 6
},
"Email": {
    "Enabled": false,
    "SmtpHost": "smtprelay.internal.example.com",
    "SmtpPort": 25,
    "From": "dashydashboard-noreply@example.com",
    "AppUrl": "http://clipvwbpod02/dashydashboard/",
    "Events": { "ScreenshotRejected": true, "AllApproved": true }
}
```

Important

Because `dotnet publish` overwrites `appsettings.Production.json`, these two sections must be re-added after *every* publish — this extends the existing "Restore `appsettings.Production.json`" step in the deployment checklist. The most reliable way to avoid re-editing the file each time is to use environment variables instead (next section); they live outside the published artifact and survive a republish.

2.4 Environment-variable alternative (recommended for per-machine values)

Every configuration key can be supplied as a machine environment variable instead of being written into the JSON file. ASP.NET Core maps a nested key to an environment variable by replacing each colon (`:`) with a double underscore (`__`):

Configuration key	Environment variable
`Screenshots:RootPath`	`Screenshots__RootPath`
`Screenshots:RetainCycles`	`Screenshots__RetainCycles`
`Email:Enabled`	`Email__Enabled`
`Email:SmtpHost`	`Email__SmtpHost`
`Email:From`	`Email__From`
`Email:Events:ScreenshotRejected`	`Email__Events__ScreenshotRejected`

Note

Prefer the environment-variable form for *per-machine* values — above all `Screenshots__RootPath`. It mirrors the existing `ConnectionStrings__Default` pattern already used on these servers: the value is set once at the machine level, lives outside the deployed artifact, and therefore survives every `dotnet publish` without any file editing. An environment variable takes precedence over the same key in `appsettings*.json`.

Chapter 3

Screenshot Storage Folder

3.1 Why it must live outside the app folder

Each deployment replaces the entire contents of the application folder (`C:\inetpub\wwwroot\DashyDashb`
If screenshots were stored inside that folder they would be deleted on the next deploy.
The storage root must therefore be a separate location, for example:

```
D:\DashyData\Screenshots
```

3.2 One-time elevated setup (per server)

This is a one-time step on each server, performed by an administrator with elevation.

1. Create the folder (substitute your chosen path):

   ```
   mkdir D:\DashyData\Screenshots
   ```

2. Grant the IIS application-pool identity *Modify* permission on the folder, applied to the folder, its subfolders, and files. Use `icacls` from an elevated Command Prompt or PowerShell. For the local development pool (`DashyDashboardPool`):

   ```
   icacls "D:\DashyData\Screenshots" /grant "IIS
   AppPool\DashyDashboardPool:(OI)(CI)M"
   ```

3. On the production server, substitute the production application pool's name for `DashyDashboardPool`. Confirm the exact pool name in IIS Manager (Application Pools) before running the command.

Note

The `icacls` flags mean: `(OI)` object inherit (files in the folder), `(CI)` container inherit (subfolders), and `M` the Modify permission set (read, write, create, delete). The app needs to create per-cycle subfolders and write image files, so Modify — not just Write — is required.

Important

The login name passed to `icacls` is derived from the application-pool name and is always prefixed `IIS AppPool\`. It is *not* a domain account and will not appear

when browsing for users — type it exactly. If the application logs "Access to the path ... is denied" when an associate uploads a screenshot, this ACL grant is the first thing to check.

3.3 Setting the path

After the folder exists, point the application at it via either:

- `Screenshots:RootPath` in `appsettings.Production.json`, *or*
- the machine environment variable `Screenshots__RootPath` (recommended — Chapter 2). The app will refuse to start if no root path is configured.

Chapter 4

Database Migrations

Two Entity Framework migrations must be applied, **in this order** (this is their merge order — the first shipped with the Tool-User-ID feature, the second with the screenshot feature):

1. `20260611112843_AddToolUserIdToUsersToolAccess` — adds `UsersToolAccess.ToolUserId` (`nvarchar(100) NULL`).
2. `20260611115259_AddScreenshotColumnsToToolCycleAttestation` — adds seven nullable columns to `ToolCycleAttestation`: `ScreenshotPath`, `ScreenshotHash`, `ScreenshotUploadedAt`, `ScreenshotStatus`, `ScreenshotReviewedBy`, `ScreenshotReviewedAt`, and `ScreenshotRejectReason`.

Important

Apply both migrations in the order listed. Applying the screenshot migration before the Tool-User-ID migration will leave the database history out of step with the deployed binaries.

4.1 Option A — apply directly with the EF tools (dev / test)

From the API project directory, with the connection string pointing at the target database:

```
$env:ConnectionStrings__Default = "Server=...;Database=...;..."
dotnet ef database update
```

`dotnet ef database update` (with no migration name) brings the database fully up to date, applying any pending migrations in order. This is appropriate for the development and test databases.

4.2 Option B — generate an idempotent SQL script for the DBA (production)

On production, the database is usually changed only by a DBA running a reviewed script. Generate an idempotent script that can be run safely regardless of which migrations are already applied:

```
dotnet ef migrations script -idempotent -output screenshots-migrations.sql
```

Hand `screenshots-migrations.sql` to the DBA to execute against `ClientsAppAttestation` in SSMS. An idempotent script wraps each migration in a guard so already-applied migrations are skipped — it is safe to run even if the first migration was applied earlier.

Note

The application's app-pool SQL login has `db_ddladmin`, so EF can also self-apply migrations on first launch in environments where that is the chosen workflow. On production, prefer the reviewed idempotent script (Option B).

Chapter 5

E-mail Notifications

E-mail is **disabled by default** (`Email:Enabled = false`). When disabled, the application behaves identically except that no mail is sent — there is no error and no impact on the request paths. Enable it only after the relay and sending mailbox are confirmed.

5.1 Events

- **ScreenshotRejected** — sent to the associate when a reviewer rejects one of their screenshots. Includes the cycle, client, tool, reviewer name, the rejection reason, and a link to re-upload in DashyDashboard.
- **AllApproved** — sent to the associate exactly once, when the last of their screenshots is approved and their attestation becomes complete.

Delivery is fire-and-forget: events are queued in-process and sent by a background consumer over an unauthenticated internal SMTP relay. A misconfigured or unreachable relay never blocks or fails a user request.

5.2 IT request template

The relay host/port and an approved sending mailbox are not Broadridge-developer-owned — request them from the messaging/IT team. A short copy-pasteable request:

```
Subject:  SMTP relay + sending mailbox for DashyDashboard

We are deploying an internal web application (DashyDashboard) on server
CLIPVWBPOD02 that needs to send automated, plain-text notification e-mails
to associates (screenshot rejected / attestation complete).

Please provide:
  1.  The internal SMTP relay hostname and port that this server may
      use to send mail (unauthenticated relay from this host is fine).
  2.  An approved "from" sending mailbox / display address for the
      From header (e.g.  dashydashboard-noreply@<domain>).

Volume is low (a handful of mails per attestation cycle).  No inbound mail
or mailbox monitoring is required.
```

5.3 Enabling

Once you have the relay details and sending mailbox:

1. Set `Email:SmtpHost`, `Email:SmtpPort`, and `Email:From` (and confirm `Email:AppUrl` points at the live site).
2. Set `Email:Enabled` to `true`.
3. Leave the per-event toggles at `true` unless a specific event is to be suppressed.

Chapter 6

Retention & Backup

6.1 Retention policy

Cycles are monthly. The policy is to retain the most recent **6** cycles' screenshots on disk (`Screenshots:RetainCycles = 6`, roughly six months).

Important

The automatic retention sweep is a *future* scheduled job and is **not yet running**. Until it ships, retention is manual.

6.2 Manual purge (until the sweep ships)

Each cycle's screenshots live under a top-level folder named for the numeric cycle ID:

```
{root}\{cycleId}\...
```

To purge, delete the `{root}\{cycleId}` folders for cycles older than the most recent six. Identify cycle IDs from `dbo.Cycles`. Deleting a folder removes only the images; the attestation rows and their stored paths remain in the database.

6.3 Backup coverage

The screenshot root folder holds attestation evidence that exists nowhere else. It should be added to the server's backup coverage so that an evidence loss is recoverable. Whether to back it up, and the retention of those backups, is an ops decision — this guide simply flags that the folder is outside the app folder and outside the database, so it is easy to miss.

Chapter 7

Local Development Box (`deploy-dev.ps1`)

The local PRV-SERVER box runs the production-mirror artifact under IIS against a local SQL instance. For the screenshot feature to work there, the same two prerequisites apply as on any server: a storage folder outside the app folder, and the configuration pointing at it. These should be added to the local `deploy-dev.ps1` (the elevated, machine-specific deploy script). The changes are described here; perform them in an elevated session — they are not part of the unelevated build.

1. **Set the machine environment variable** so it survives every republish, mirroring the existing `ConnectionStrings__Default` line:

```
[Environment]::SetEnvironmentVariable(
    "Screenshots__RootPath", "D:\DashyData\Screenshots", "Machine")
```

2. **Create the folder and grant the local app-pool identity Modify** (idempotent):

```
New-Item -ItemType Directory -Force "D:\DashyData\Screenshots"
icacls "D:\DashyData\Screenshots" /grant "IIS
AppPool\DashyDashboardPool:(OI)(CI)M"
```

Note

`deploy-dev.ps1` already performs the analogous elevated work for the connection string (setting `ConnectionStrings__Default` as a machine variable) and for IIS. Adding the two steps above keeps the local box self-contained and republish-proof. The local pool is `DashyDashboardPool`; on production substitute the real pool name.

Chapter 8

Future Phase

The following are documented as the planned *next* phase. They reuse the e-mail plumbing (the queue + send split in `EmailService`) and the storage service's `Delete` method that already exist, so no new infrastructure is required — only a scheduler.

- **Scheduled `BackgroundService`.** A single hosted service that wakes on a timer (e.g. daily) and performs the periodic tasks below. It composes mail via the existing `EmailService` and enqueues it the same way the request paths do.
- **Due-date reminder mails.** Seven days before a cycle's `DueDate`, mail associates who have not yet completed their attestation.
- **Manager pending-approvals digest.** A periodic summary to each reviewer listing the screenshots awaiting their approval.
- **Retention sweep.** Automatically delete `{root}\{cycleId}` folders for cycles older than `Screenshots:RetainCycles`, removing the manual purge step in Chapter 6.

Note

None of these are built in the current release. They are listed so that the configuration keys (`Screenshots:RetainCycles`, the `Email` section) and the storage layout described in this guide are understood as forward-compatible with the next phase.